

附录 A

Transformer模型术语

过去几十年里，产生了卷积神经网络(CNN)、递归神经网络(RNN)和其他更多类型的人工神经网络(ANN)。它们都使用一些共同术语。

Transformer 模型引入了一些新术语，并稍微改变了现有术语的定义。本附录将介绍这些新术语以及一些现有深度学习术语在应用于 Transformer 模型时的定义。

Transformer 模型架构解决了 RNN 难以并行化的缺陷，从而带领深度学习进入工业化时代。Transformer 这种架构十分容易进行硬件优化。例如 Google 针对深度学习设计的优化硬件 TPU 结合 Transformer 的堆叠结构就能发挥强大威力。

Transformer 模型对算力要求很高，所以在设计时需要考虑硬件因素。因此，设计 Transformer 架构一开始就要考虑软件和硬件方面的优化。

本附录将介绍 Transformer 引入的新术语。

堆叠(Stack)

与传统深度学习模型不同，Transformer 等现代深度学习模型将相同大小的层堆叠在一起，如图 A.1 所示。我们从下到上依次运行堆叠中的层。堆叠可以应用于编码器或解码器。



图 A.1 将多个层进行堆叠

随着堆叠攀升，Transformer 可以学习和看到越来越多的知识。每一层都将其学到的知识传递到更高一层，就像我们的记忆一样。

做个比喻，一个堆叠就像纽约的帝国大厦。在堆叠底部，你看不到太多风景。但是，随着你在堆叠里攀升，可以看到越来越多、越来越远的地方。最后，到了堆叠顶部，可以欣赏到整个纽约市区的美景！

子层(Sublayer)

Transformer 模型的每层都包含如图 A.2 所示的子层。每一层的每个子层都具有相同的结构，从而有助于硬件优化(并行化)。



图 A.2 每层包含两个子层

原始 Transformer 模型包含两个从下到上依次运行的子层：

- 针对 NLP 和硬件优化设计的自注意力子层
- 做了一些调整的传统前馈神经网络子层

注意力头(Attention heads)

自注意力子层分为 n 个独立和相同的层，这些层称为自注意力头。原始 Transformer 模型包含八个自注意力头。

图 A.3 用八个并行处理的 CPU 比作自注意力头，来表示 Transformer 的工业化设计结构如何适用于硬件优化(并行化)。

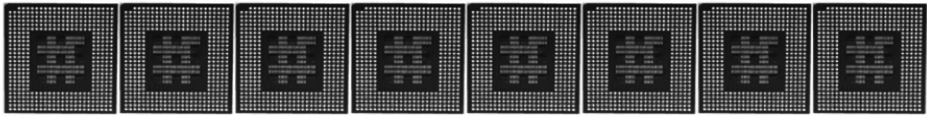


图 A.3 自注意力子层包含多个自注意力头

至此，我们可以看到 Transformer 架构既满足了 NLP 需求，又满足了硬件优化需求。

附录 B

Transformer模型的硬件约束

如果没有硬件优化，Transformer 模型的训练就会旷日持久，不大可能流行。内存和磁盘管理设计仍然很关键。然而，计算能力才是最关键的先决条件。如果没有 GPU，几乎不可能训练第 2 章描述的原始 Transformer。GPU 是高效 Transformer 模型的核心。

本附录将通过三个部分带你了解 GPU 的重要性：

- Transformer 的架构和扩展
- CPU 与 GPU
- 如何在 PyTorch 调用 GPU

Transformer 的架构和扩展

每分析一次 d_{model} ，我们只能得到一个观点。如果不能并行计算，我们需要相当多的计算时间才能找到所有观点。

有一种更好的方法是将每个单词的 $d_{\text{model}} = 512$ 维分成 8 块(即 $d_k = 64$ 维)。

然后，我们可以并行运行 8 个“头”以加快训练速度，并获得 8 个不同的、体现了每个单词与另一个单词关系的表示子空间，如图 B.1 所示。

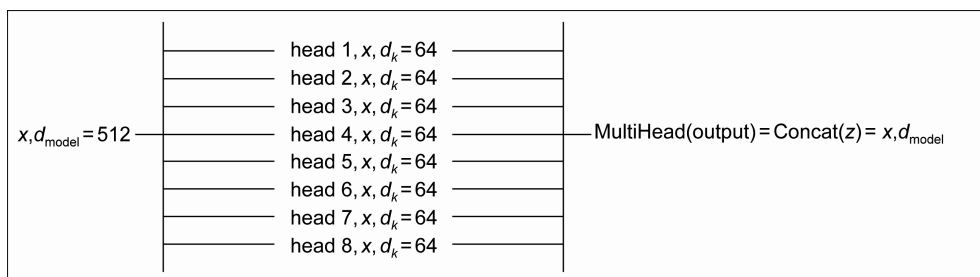


图 B.1 多头表示

可以看到有 8 个注意力头在并行运行。

可以很容易地理解多个注意力头并行学习 8 种不同观点的优点。然而仅仅是并行学习是不够的，毕竟 CPU 也可以并行运算，但是使用 CPU 进行并行运算会比使用

GPU 并行运算慢很多，所以我们还是需要针对 GPU 做硬件优化。

Brown et al. (2020), in Language Models are Few-Shot Learners(<https://arxiv.org/abs/2005.14165>)论文提到，他们设计 GPT 模型的时候，专门针对 GPU 做了硬件优化。

他们将模型沿着深度和宽度维度划分到 GPU 上，以最小化节点之间的数据传输。每个模型的精确架构参数是基于计算效率和负载平衡在 GPU 上的模型布局来选择的。

每个 Transformer 模型在其结构(编码器和解码器)和规模上都有所不同，但都针对 GPU 做了硬件优化。接下来进一步了解要针对 GPU 做哪些硬件优化，GPU 和 CPU 相比，特别之处在哪里。

GPU 特别之处

第 2 章除了提到多头注意力专为并行计算而设计之外，还提到了注意力具体定义为“缩放点积注意力”，可使用下面的等式表示：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

可得出以下结论：

- 多头注意力机制专为并行计算而设计
- 多头注意力机制基于矩阵乘法(matmul)

GPU 针对并行处理进行了优化

CPU(中央处理单元)针对串行处理进行了优化。但是，如果我们通过串行处理来运行注意力头，那么训练一个高效的 Transformer 模型需要很长时间。可以使用 CPU 训练非常小的仅用于教学目的的 Transformer。但我们无法使用 CPU 来训练出最先进的模型。

GPU(图形处理单元)针对并行处理进行了优化。所以 Transformer 模型更适合使用 GPU 而不是 CPU。

GPU 针对矩阵乘法进行了优化

GPU 也针对矩阵乘法进行了优化。例如，NVIDIA 的 GPU 中包含了张量核心(tensor cores)，可以加速矩阵运算。许多人工智能算法，包括 Transformer 模型，都使用了矩阵运算。因此，NVIDIA 的 GPU 在硬件优化方面具有很大优势。更多信息参见：

- <https://blogs.nvidia.com/blog/2009/12/16/whats-the-difference-between-a-CPU-and-a-GPU/>
- <https://www.nvidia.com/en-us/data-center/tesla-p100/>

Google 的 Tensor Processing Unit(TPU)是类似于 NVIDIA GPU 的处理器。在使用

TPU 时，TensorFlow 会优化张量的使用。

- 有关 TPU 的更多信息，请参见 <https://cloud.google.com/tpu/docs/tpus>。
- 有关 TensorFlow 中张量的更多信息，请参阅 <https://www.tensorflow.org/guide/tensor>。

BERT_{BASE} (110M 参数)最初使用了 16 个 TPU 芯片训练。BERT_{LARGE} (340M 参数)使用了 64 个 TPU 芯片训练。有关训练 BERT 的更多信息，请参阅 <https://arxiv.org/abs/1810.04805>。

至此，我们已经确定，Transformer 的架构确实能够充分利用 GPU、TPU 的硬件性能。接下来我们将讲述如何在代码中调用 GPU 进行运算。

在代码中调用 GPU

可通过 PyTorch 等框架来调用 GPU 进行运算。PyTorch 有一个称为“张量(tensor)”的数据结构，类似于 TensorFlow 中的张量。张量看起来可能类似于 NumPy 中的 np.arrays()。然而，与 NumPy 不同的是，张量可以利用 GPU 的并行处理能力。

PyTorch 等框架通过张量打开了 GPU 上分布式数据的大门：https://pytorch.org/tutorials/intermediate/ddp_tutorial.html。

在第 3 章的 BERT_Fine_Tuning_Sentence_Classification_GPU.ipynb，我们使用 CUDA(Compute Unified Device Architecture)与 NVIDIA GPU 进行通信。CUDA 是 NVIDIA 使用 GPU 进行通用计算的平台。我们可以在代码中调用特定的 CUDA 指令。更多信息请参阅 <https://developer.nvidia.com/cuda-zone>。

第 3 章中，我们使用 CUDA 指令将模型和数据传输到 NVIDIA GPU。可以在 PyTorch 使用如下指令来指定我们希望使用的设备：torch.device。

更多信息请参阅 <https://pytorch.org/docs/stable/notes/cuda.html>。

接下来我们将介绍如何在 PyTorch 和其他程序调用 GPU，将介绍如何选择 GPU，如何并行处理数据，如何将模型加载到 GPU 上以及将训练和验证数据批量添加到 GPU。以下代码来自 BERT_Fine_Tuning_Sentence_Classification_GPU.ipynb：

- 选择 GPU(单元格 3)

可使用以下代码检查是否可通过 CUDA 调用 NVIDIA GPU。如果不可以，将调用 CPU：

```
device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")
!nvidia-smi
```

- 并行处理数据(单元格 16)

如果有多个 GPU 可用，则可以使用以下代码调用多个 GPU 进行并行计算：

```
model = BertForSequenceClassification.from_pretrained("bert-baseuncased",
num_labels=2)
model = nn.DataParallel(model)
```

- 将模型加载到设备(单元格 16)
可使用以下代码将模型加载到 GPU 上:

```
model.to(device)
```

- 将训练和验证数据批量添加到设备(单元格 20)
可使用以下代码将训练和验证数据批量添加到可用的 GPU 上(从 1 到 n):

```
# Add batch to GPU
batch = tuple(t.to(device) for t in batch)
```

接下来将详细描述如何在 Google Colab 调用 GPU。

在 Google Colab 调用 GPU

将使用第 3 章的 `BERT_Fine_Tuning_Sentence_Classification_GPU.ipynb` 来讲述如何在 Google Colab 调用 GPU。

将在以下情况下运行:

- 在 Google Colab 免费版使用 CPU
- 在 Google Colab 免费版使用 GPU
- 在 Google Colab 专业版使用 GPU

在 Google Colab 免费版使用 CPU

如前所述,几乎不可能使用 CPU 对具有数百万或数十亿参数的 Transformer 模型进行微调或训练。CPU 不擅长并行处理。而 Transformer 模型需要大量的并行处理。

打开 Runtime 菜单中的 Change Runtime Type 子菜单,这里有三个选项:None (CPU)、GPU、TPU。

我们首先选择 None(CPU),如图 B.2 所示。

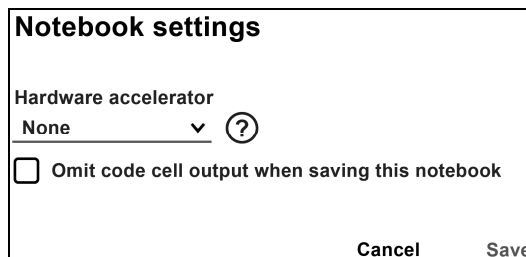


图 B.2 选择 CPU

然后开始训练,我们发现一开始就很慢,如图 B.3 所示。

```
Epoch : 0% | | 0/4 [00:00<?, ?it/s]
```

图 B.3 训练循环

训练了 15 分钟后，进度基本上不变，好像什么都没有发生一样。

CPU 不擅长并行处理。而 Transformer 模型需要大量的并行处理，因此我们需要 GPU。

在 Google Colab 免费版使用 GPU

这次我们选择 GPU，如图 B.4 所示。

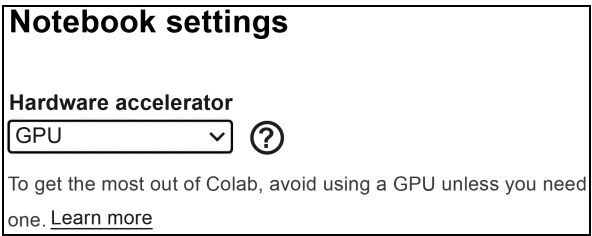


图 B.4 选择 GPU

Google Colab 免费版提供了 NVIDIA K80 和 CUDA 11.2，如图 B.5 所示。

NVIDIA-SMI 495.44				Driver Version: 460.32.03				CUDA Version: 11.2			
GPU Name		Persistence-M		Bus-Id		Disp.A		Volatile Uncorr. ECC			
Fan	Temp	Perf	Pwr:Usage/Cap	Memory-Usage		GPU-Util		Compute M.		MIG M.	
0	Tesla K80		Off	00000000:00:04.0 Off						0	
N/A	39C	P8	27W / 149W	0MiB / 11441MiB				0%		Default N/A	
Processes:											
GPU	GI	CI	PID		Type	Process name			GPU Memory Usage		
	ID	ID									
No running processes found											

图 B.5 使用了 NVIDIA K80 GPU

我们只花了 20 分钟就完成了训练。不过，在本书撰写时，Google Colab VM 只提供一个 GPU，没有提供多个 GPU。GPU 很贵。但是训练起来也很快，如图 B.6 所示。

```
Epoch: 0%|          | 0/4 [00:00<?, ?it/s]Train los
Epoch: 25%|██████    | 1/4 [04:58<14:56, 299.00s/it]
Train loss: 0.30048875815208026
Epoch: 50%|██████████| 2/4 [09:58<09:58, 299.42s/it]
Train loss: 0.1783793037950498
Epoch: 75%|███████████| 3/4 [14:58<04:59, 299.55s/it]
Train loss: 0.11217724044973425
Epoch: 100%|███████████| 4/4 [19:58<00:00, 299.57s/it]
```

图 B.6 使用 K80 GPU 的训练循环

接下来我们看看收费的 Google Colab 专业版的 GPU 效果如何。

在 Google Colab 专业版使用 GPU

Google Colab 专业版提供了一个 NVIDIA P100 GPU，如图 B.7 所示。这个 GPU 很强大，因为原始 Transformer 也只是用 8 个 NVIDIA P100 训练的。如 Vaswani et al.(2017), Attention is All you Need 论文里所述，他们使用 8 个 GPU 花了 12 个小时训练完具有 106×65 个参数的原始 Transformer 模型。

NVIDIA-SMI 495.44												Driver Version: 460.32.03				CUDA Version: 11.2			
GPU		Name		Persistence-M				Bus-Id				Disp.A		Volatile Uncorr. ECC					
Fan		Temp		Perf		Pwr:Usage/Cap				Memory-Usage				GPU-Util		Compute M.		MIG M.	
0		Tesla		P100-PCIE...				Off				00000000:00:04.0 Off				0			
N/A		41C		P0		28W / 250W				2MiB / 16280MiB				0%		Default		N/A	

图 B.7 Google Colab 专业版配备了 P100 GPU

可以看到，训练循环时间大大减少，只需要不到 10 分钟(免费版的 NVIDIA K80 GPU 需要 20 分钟)，如图 B.8 所示。

Epoch: 0%		0/4 [00:00<?, ?it/s]Train lo
Epoch: 25%		1/4 [01:35<04:47, 95.71s/it]
Train loss: 0.3125095507168671		
Epoch: 50%		2/4 [03:11<03:11, 95.57s/it]
Train loss: 0.18029312002646478		
Epoch: 75%		3/4 [04:46<01:35, 95.51s/it]
Train loss: 0.11255507657296678		
Epoch: 100%		4/4 [06:22<00:00, 95.53s/it]

图 B.8 使用 P100 GPU 的训练循环

附录 C

使用GPT-2进行文本补全

第 7 章介绍了使用 GPT-2 进行文本补全，本附录将进行详细说明。本附录描述了如何实现一个 GPT-2 Transformer 模型进行文本补全。

可直接在第 7 章阅读此笔记本的用法，或在本附录构建程序并运行它，以更深入地了解 GPT 模型的工作原理。

我们将使用本书配套 GitHub 代码存储库中的 `AppendixIII/OpenAI_GPT_2.ipynb`，将下载 GPT-2 345M Transformer 模型，并与之交互。我们将输入上下文句子并分析 Transformer 生成的文本，目标是看看它是如何创建新内容的。

本节分为九个步骤。我们需要使用 Google Colab 打开 `OpenAI_GPT_2.ipynb`。该笔记本位于本书配套 GitHub 代码存储库的 `AppendixIII` 目录中。你会注意到，该笔记本也分为与本节相同的九个步骤和对应的单元格。

我们将逐个单元格运行笔记本。这个过程很乏味，但随之产生的结果是可喜的。在之前的章节中，我们看到只需要几行代码就可以运行 GPT-3 引擎，这样做虽然简单快捷，却让你失去了解 GPT 模型工作原理的机会。本附录将提供这么一个机会：不用这些优化代码，也能了解 GPT-2 模型的工作原理。

Hugging Face 有一个封装好 GPT-2 模型的包装器，是一个很有用的 OpenAI API 替代品。但如前所述，本附录的目标不是通过包装器以避免 GPT-2 模型的底层组件的复杂性，而是探索它们以了解 GPT-2 模型的工作原理！

最后要强调的是，我们是在底层运行 GPT-2 模型，而不是只通过一行代码调用来获得结果。这就是为什么我们要避免使用预打包版本(OpenAI GPT-3 API、Hugging Face 包装器等)。我们是要从头开始了解 GPT-2 的架构。因此，在这个过程中，你可能看到一些弃用消息，也许比较讨厌。但这个努力是值得的，因为可通过这个努力成为一个工业 4.0 的 AI 大师。

先从激活 GPU 开始。

步骤 1：激活 GPU

必须激活 GPU 来训练 GPT-2 345M Transformer 模型。

请单击菜单 Runtime→Change runtime type 进入如图 C.1 所示的界面，并按图示激活 GPU，以充分利用 VM。

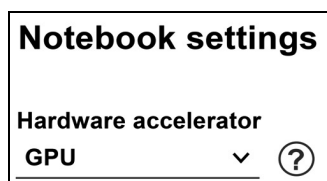


图 C.1 激活 GPU

可以看到，激活 GPU 是获得更好性能的先决条件，从而令我们能够进入 GPT Transformer 模型的世界。接下来我们复制 OpenAI GPT-2 代码存储库。

步骤 2：克隆 OpenAI GPT-2 代码存储库

OpenAI 现在仍然允许我们下载 GPT-2。OpenAI 可能在将来不允许我们下载了，或者我们可能找到了更好的替代品。在这一点上，只能说 Transformer 的发展是十分迅速的，以至于没有人能预见将来会发展得如何，即使是主流研究实验室也不敢对此下定论。

先在 VM 上克隆 OpenAI GPT-2 的 GitHub 代码存储库：

```
#@title Step 2: Cloning the OpenAI GPT-2 Repository
!git clone https://github.com/openai/gpt-2.git
```

复制结束后，你应该可在文件管理器中看到如下代码存储库，如图 C.2 所示。

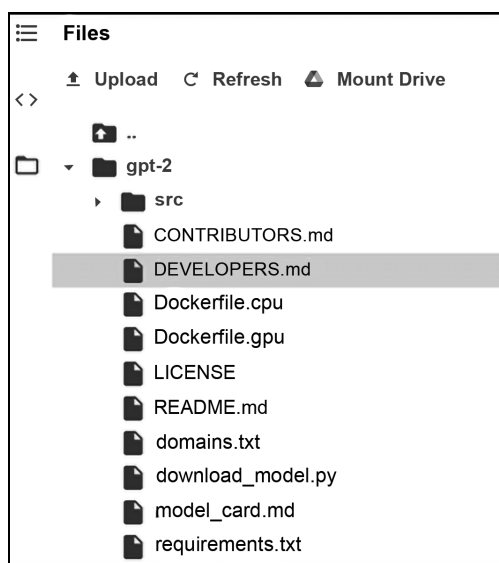


图 C.2 复制到 VM 上的 GPT-2 代码存储库

单击 src，你会看到我们运行 OpenAI GPT-2 模型所需的 Python 文件，如图 C.3 所示。

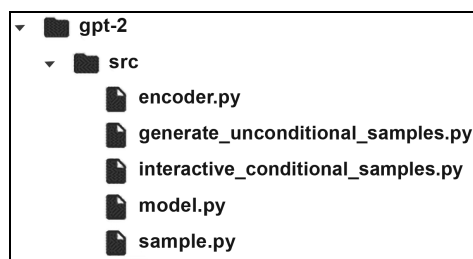


图 C.3 用于运行 GPT-2 模型的 Python 文件

可看到我们没有安装训练模型相关的东西。我们将在附录 D 安装它们。
接下来安装必需的库。

步骤 3：安装必需的库

通过以下代码安装必需的库：

```
#@title Step 3: Installing the requirements
import os # when the VM restarts import os necessary
os.chdir("/content/gpt-2")
!pip3 install -r requirements.txt
```

注意，如果逐个单元格运行，你可能需要重启 VM，以再次导入 os 模块。

我们所需要的库如下：

- Fire 0.1.3 用于生成命令行界面(CLI)
- regex 2017.4.5 用于处理正则表达式
- Requests 2.21.0 用于处理 HTTP 请求
- tqdm 4.31.1 用于在循环时展示进度

Google Colab 可能要求重启笔记本。

这时请不要重启。请等我们检查完 TensorFlow 的版本后再重启。

步骤 4：检查 TensorFlow 的版本

OpenAI 提供的 GPT-2 345M Transformer 模型所使用的 TensorFlow 版本是 1.x。这点会导致在运行程序时出现几个警告。请忽略它们，并用我们弱小的机器全速运行庞大的 GPT 模型。

到了 2020 年，GPT 模型已达到 1750 亿个参数(GPT-3)，这点令我们无法在没有超级计算机的情况下自行训练 GPT 模型。将来参数的数量只会不断增加。

各大企业巨头的研究实验室，如 Facebook AI 和 OpenAI，以及 Google Research/Brain，正在加速往超大型 Transformer 模型方向走，并留下他们的学习成果

让我们学习和摆脱困境。但是，遗憾的是，他们没有时间回过头来更新他们共享过的所有模型。幸好我们仍然有这个笔记本！

TensorFlow 的最新版本是 2.x。但一些较旧的程序还依赖于 1.x。这就是为什么 Google Colab VM 同时预装了 TensorFlow 1.x 和 TensorFlow 2.x 版本的原因之一。

这个笔记本将使用 TensorFlow 1.x:

```
#@title Step 4: Checking the Version of TensorFlow
#Colab has tf 1.x and tf 2.x installed
#Restart runtime using 'Runtime' -> 'Restart runtime...'
%tensorflow_version 1.x
import tensorflow as tf
print(tf.__version__)
```

应该会得到以下输出结果:

```
TensorFlow 1.x selected.
1.15.2
```

无论版本是否展示为 1.x，都请重新运行该单元格以确保现在选择了 1.x 版本，然后重启 VM。在继续之前重新运行该单元格。



如果在该过程中遇到 TensorFlow 错误，请重新运行该单元格，重启 VM，然后重新运行该单元格。

记住！每次重启 VM 之后都要重新运行该单元格。因为 VM 默认为 TensorFlow 2.x 版本。

现在我们准备下载 GPT-2 模型。

步骤 5：下载 GPT-2 345M Transformer 模型

现在下载 GPT-2 345M Transformer 模型：

```
#@title Step 5: Downloading the 345M parameter GPT-2 Model
# run code and send argument
import os # after runtime is restarted
os.chdir("/content/gpt-2")
!python3 download_model.py '345M'
```

模型会下载到以下目录：

```
/content/gpt-2/models/345M
```

下载成功后，可在文件管理器看到如图 C.4 所示的目录和文件。

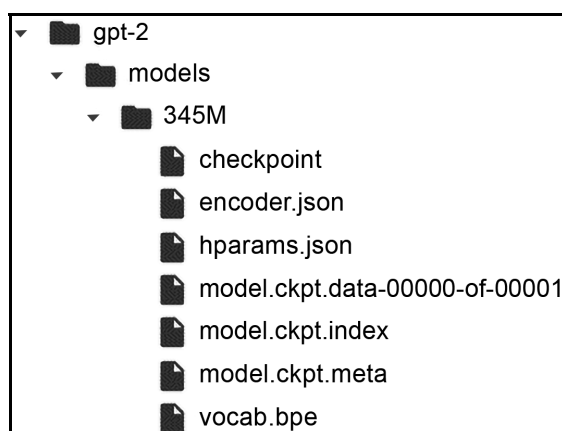


图 C.4 GPT-2 345M Transformer 模型的目录和文件

其中 hparams.json 文件包含了 GPT-2 模型的定义：

- n_vocab: 50257, 模型词表的大小
- n_ctx: 1024, 上下文大小
- n_embd: 1024, 嵌入大小
- n_head: 16, 注意力头数
- n_layer: 24, 层的数量

encoder.json 和 vocab.bpe 包含词元化词表和 BPE 单词对。如有必要, 请花几分钟时间阅读第 4 章。

checkpoint 文件包含了 checkpoint 处的训练参数。例如, 它可包含 1000 步的训练参数, 正如我们将在附录 4 步骤 9 所做的那样。

与 checkpoint 文件一起保存的还有以下三个重要文件：

- model.ckpt.meta 描述了模型的图结构(包括 GraphDef、SaverDef 等)。可通过 `tf.train.import_meta_graph([path]+'model.ckpt.meta')` 获取相关信息。
- model.ckpt.index 是一个字符串表。key 包含张量的名称, value 为 BundleEntryProto 类型, 该类型包含了张量的元数据。
- model.ckpt.data 包含 TensorBundle 集合中所有变量的值。

我们已经下载了模型。但在激活模型前还需要完成一些中间步骤。

步骤 6-7a: 中间步骤

本节将完成步骤 6、7 和 7a, 它们是通往步骤 8 的中间步骤; 我们将通过这些步骤定义并激活模型。

步骤 6: 打印时使用 UTF 编码:

```
#@title Step 6: Printing UTF encoded text to the console
!export PYTHONIOENCODING=UTF-8
```

步骤 7: 将当前工作目录改为/content/gpt-2/src:

```
##title Step 7: Project Source Code
import os # import after runtime is restarted
os.chdir("/content/gpt-2/src")
```

现在已准备好与 GPT-2 模型进行交互。可以直接运行这个模型,就像我们将在附录 D 所做的那样。但是,本节将介绍代码的主要方面。

步骤 7a: 导入与模型交互所需的模块:

```
##title Step 7a: Interactive Conditional Samples (src)
#Project Source Code for Interactive Conditional Samples:
# /content/gpt-2/src/interactive_conditional_samples.py file
import json
import os
import numpy as np
import tensorflow as tf
```

现在我们已经完成了激活模型所需的中间步骤了。

步骤 7b-8: 导入和定义模型

现在将使用 `interactive_conditional_samples.py` 激活与模型的交互。

需要导入/content/gpt-2/src 中的三个模块:

```
import model, sample, encoder
```

这三个模块的作用是:

- `model.py` 定义了模型的结构: 超参数、多头注意力运算 `tf.matmul`、激活函数和其他所有属性。
 - `sample.py` 处理交互并控制生成的样本。它确保词元更有意义。
`softmax` 值有时可能模糊,就像在低清晰度下查看图像一样。`sample.py` 包含一个名为 `temperature` 的变量,可使值更清晰,增加更高的概率并软化较低的概率。
`sample.py` 可激活 Top-k 采样。Top-k 采样对预测序列的概率分布进行排序。分布在头部的较高概率值被过滤到第 k 个词元。包含较低概率的尾部被排除,防止模型预测出低质量的词元。
`sample.py` 还可激活用于语言建模的 Top-p 采样。Top-p 采样不对概率分布进行排序。相反,它选择具有高概率的单词,直到该子集的概率之和或可能序列的核心超过 p。
 - `encoder.py` 使用定义的模型、`encoder.json` 和 `vocab.bpe` 对样本序列进行编码。它包含 BPE 编码器和文本解码器。
- 可双击打开这些程序以进一步探索它们。

`interactive_conditional_samples.py` 将调用与模型交互所需的函数来初始化以下信息：来自 `model.py` 的定义模型的超参数，以及来自 `sample.py` 的样本序列参数。它将使用 `encode.py` 对序列进行编码和解码。

然后，`interactive_conditional_samples.py` 将恢复在本节步骤 5 中定义的 checkpoint 数据。

可以双击 `interactive_conditional_samples.py` 文件来探索它，并尝试调整它的参数：

- `model_name` 是模型的名称，"124M"或"345M"，具体名称取决于 `models_dir`。
- `models_dir` 定义了包含模型的目录。
- `seed` 设置一个随机整数作为随机生成器的种子。种子可以用来复现结果。
- `nsamples` 是要返回的样本数量。如果设置为 0，它将继续生成样本，直到你双击单元格的 `run` 按钮或按下 `Ctrl + M`。
- `batch_size` 确定批量的大小，对内存和速度有影响。
- `length` 是生成文本的词元数量。如果设置为 `none`，则依赖于模型的超参数。
- `temperature` 确定 Boltzmann 分布的级别。如果 `temperature` 较高，生成文本将更加随机。如果 `temperature` 较低，结果将更加确定。
- `top_k` 控制每个步骤中考虑的 Top-k 词元数量。0 表示没有限制。建议值为 40。
- `top_p` 控制 Top-p。

本例中使用如下参数值(对应了后面步骤 9 中的代码)：

- `model_name = "345M"`
- `seed = None`
- `nsamples = 1`
- `batch_size = 1`
- `length = 300`
- `temperature = 1`
- `top_k = 0`
- `models_dir = '/content/gpt-2/models'`

这些参数将影响模型的行为，决定了模型如何根据上下文输入进行条件化，并生成文本补全序列。首先使用默认值运行笔记本。然后，可通过双击程序、编辑并保存它来更改代码的参数。每次重新启动 VM 时，你所做修改都会被清空。如果你希望创建交互场景，请保存程序并重新加载它。

现在一切都准备好了，终于可以与模型交互了。

步骤 9：与 GPT-2 交互

本节将与 GPT-2 345M 模型进行交互。

程序运行时系统会显示很多消息，不过这些消息主要与 Google Colab 维护

TensorFlow 1.x 相关，与我们这个笔记本无关。不过这个笔记本可能有一天会过时，到时候我们可能不得不使用 GPT-3 引擎，或者不得不使用 Hugging Face GPT-2 包装器(这个包装器到时也可能被弃用)。

现在，还能使用 GPT-2，所以让我们与模型交互吧！

要与模型交互，请运行以下单元格：

```
#@title Step 9: Interacting with GPT-2
interact_model('345M',None,1,1,300,1,0,'/content/gpt-2/models')
```

系统将提示你输入一些上下文，如图 C.5 所示。

prompt >>>

图 C.5 输入文本补全的上下文

可尝试任何类型的上下文，因为这是标准的 GPT-2 模型。

可尝试 Emmanuel Kant 写的一句话：

```
Human reason, in one sphere of its cognition, is called upon to consider
questions, which it cannot decline, as they are presented by its own nature,
but which it cannot answer, as they transcend every faculty of the mind.
```

按 Enter 键生成文本。输出将是相对随机的，因为 GPT-2 模型没有在我们自己的数据集上训练过，而且现在我们运行的是随机模型。

我们来看看 GPT 模型生成的前几行：

```
"We may grant to this conception the peculiarity that it is the only
causal logic.
In the second law of logic as in the third, experience is measured at its
end: apprehension is afterwards closed in consciousness.
The solution of scholastic perplexities, whether moral or religious, is
not only impossible, but your own existence is blasphemous."
```

如果需要停止单元格，请双击单元格最左边的 run 按钮。

也可按 Ctrl+M 停止生成文本，但它可能将代码转换为文本，并且你必须将其复制回代码单元格。

输出很丰富。我们可以观察到几点：

- 向模型提供了一些上下文信息，并基于该信息生成了输出文本。
- 上下文是用来指导模型生成输出的信息，模型通过学习上下文来决定要生成的内容，而不需要调整模型的参数。
- 文本补全指的是根据给定上下文条件来完成或延续文本。这一特性使得 Transformer 等模型能在不进行微调的情况下生成连贯的文本。
- 与语义相关的角度看，生成的输出可能会更有趣。这意味着根据上下文条件生成的文本可能会更富有含义。

- 从语法角度看，生成的输出文本很具说服力，即它们在语法结构上是合理的。接下来的附录 D，我们将使用自定义数据集训练 GPT-2 模型。

参考资料

- OpenAI GPT-2 GitHub 代码存储库：<https://github.com/openai/gpt-2>
- N Shepperd 的 GitHub 代码存储库：<https://github.com/nshepperd/gpt-2>

附录 D

使用自定义数据集训练 GPT-2模型

本附录与第7章相关，描述了如何使用自定义数据集训练GPT-2模型以自定义文本补全。

本附录将介绍如何通过 12 个步骤构建、训练 GPT-2 模型以及与自定义文本交互。

打开位于本书配套 GitHub 代码存储库 AppendixIV 目录中的 Training_OpenAI_GPT_2.ipynb。你会注意到，该笔记本也分为与本附录相同的 12 个步骤和单元格。

我们将逐个单元格运行笔记本。这个过程很乏味，但随之产生的结果是可喜的。我们没有使用 GPT-3 API 或 Hugging Face 包装器。

我们要亲自动手看看模型是如何构建和训练的。在这个过程中，你可能看到一些弃用消息，因为我们需要进入模型内部，而不是调用包装器或 API。这个过程可能会比较令人讨厌甚至痛苦，但是，这种努力是值得的。

先从激活 GPU 开始。

训练 GPT-2 语言模型

本节将对自定义数据集编码并训练 GPT-2 模型。然后，将与我们的自定义模型进行交互。将使用第 4 章的 kant.txt 数据集。

打开笔记本并逐个单元格运行。

步骤 1：先决条件

本节中提到的内容可在本书配套 GitHub 代码存储库的 AppendixIV 目录中找到：

- 请如附录 C 中步骤 1 所述，单击菜单 Runtime->Change runtime type 激活 GPU。

- 使用内置文件管理器将以下 Python 文件上传到 Google Colab: train.py、load_dataset.py、encode.py、accumulate.py、memory_saving_gradients.py。
- 这些文件最初来自 N Shepperd 的 GitHub 存储库: <https://github.com/nshepperd/gpt-2>。但可从本书配套 GitHub 存储库的 AppendixIV\gpt-2-train_files 目录下载这些文件。
- N Shepperd 的 GitHub 存储库提供了训练 GPT-2 模型所需的文件。我们不会复制 N Shepperd 的存储库。相反,将复制 OpenAI 的存储库,然后从 N Shepperd 的存储库中添加我们需要的五个训练文件。
- 使用内置文件管理器将 dset.txt 上传到 Google Colab。该数据集被命名为 dset.txt,以便你在阅读本附录后可以替换其内容,而不需要修改程序以适应你的自定义输入。
- 我们的数据集位于本书配套 GitHub 存储库的 AppendixIV\gpt-2-train_files 目录中。它是第 4 章所使用的 kant.txt 数据集。

接下来将执行训练过程的初始步骤。

步骤 2~6: 训练过程的初始步骤

本节将简要介绍步骤 2 到 6,因为我们已经在附录 C 详细描述过它们了。然后,将数据集和模型复制到项目目录中。

程序现在复制 OpenAI 的 GPT-2 存储库,而不是 N Shepperd 的存储库:

```
##title Step 2: Cloning the OpenAI GPT-2 Repository
#!git clone https://github.com/nshepperd/gpt-2.git
!git clone https://github.com/openai/gpt-2.git
```

现在已经从 N Shepperd 的目录中上传了训练 GPT-2 模型所需的数据。

接下来安装所需的依赖项:

```
##title Step 3: Installing the requirements
import os #when the VM restarts import os necessary
os.chdir("/content/gpt-2")
!pip3 install -r requirements.txt
```

这个笔记本需要 toposort,这是一种拓扑排序算法:

```
! pip install toposort
```



安装完依赖项后不要重新启动笔记本。相反,请等到你检查 TensorFlow 版本后重新启动 VM,从而保证只重新启动 VM 一次。之后如有必要才重新启动它。这么做很乏味,但很值得。

现在我们检查 TensorFlow 的版本,以确保运行的是 tf 1.x 版本:

```

#@title Step 4: Checking TensorFlow version
#Colab has tf 1.x , and tf 2.x installed
#Restart runtime using 'Runtime' -> 'Restart runtime...'
%tensorflow_version 1.x
import tensorflow as tf
print(tf.__version__)

```

无论版本是否展示为 1.x，都请重新运行该单元格以确保我们现在选择了 1.x 版本，然后重启 VM。在继续之前重新运行该单元格。

然后将下载 117M 参数的 GPT-2 模型：

```

#@title Step 5: Downloading 117M parameter GPT-2 Model
# run code and send argument
import os # after runtime is restarted
os.chdir("/content/gpt-2")
!python3 download_model.py '117M' #creates model directory

```

将数据集和 117M 参数的 GPT-2 模型复制到 src 目录中：

```

#@title Step 6: Copying the Project Resources to src
!cp /content/dset.txt /content/gpt-2/src/
!cp -r /content/gpt-2/models/ /content/gpt-2/src/

```

目标是将训练模型需要的所有资源都组织在 src 项目目录中。

现在我们将浏览 N Shepperd 的训练文件。

步骤 7：N Shepperd 的训练文件

将使用的训练文件来自 N Shepperd 的 GitHub 存储库。我们在步骤 1 上传了它们。现在将它们复制到项目目录中：

```

#@title Step 7: Copying the N Shepperd Training Files
#Referfence GitHub repository: https://github.com/nshepperd/gpt-2
import os # import after runtime is restarted
!cp /content/train.py /content/gpt-2/src/
!cp /content/load_dataset.py /content/gpt-2/src/
!cp /content/encode.py /content/gpt-2/src/
!cp /content/accumulate.py /content/gpt-2/src/
!cp /content/memory_saving_gradients.py /content/gpt-2/src/

```

训练文件现已准备就绪，可以启动了。现在使用 encode.py 对数据集进行编码。

步骤 8：对数据集进行编码

在训练前，必须对数据集进行编码。将使用 encode.py 对数据集进行编码，可在 Google Colab 双击 encode.py 来查看具体代码。

encode.py 通过调用 load_dataset.py 中的 load_dataset 函数来加载 dset.txt：

```
from load_dataset import load_dataset
.../...
chunks = load_dataset(enc, args.in_text, args.combine, encoding=args.
encoding)
```

`encode.py` 还加载了 OpenAI 的编码程序 `encode.py` 对数据集进行编码：

```
import encoder
.../...
enc = encoder.get_encoder(args.model_name, models_dir)
```

编码后的数据集以 NumPy 数组的形式保存在 `out.npz` 中。这个文件是一个由编码器生成的 NumPy zip 存档：

```
import numpy as np
np.savez_compressed(args.out_npz, *chunks)
```

然后通过以下代码将数据集加载、编码，并保存到 `out.npz`：

```
#@title Step 8:Encoding dataset
import os # import after runtime is restarted
os.chdir("/content/gpt-2/src/")
model_name="117M"
!python /content/gpt-2/src/encode.py dset.txt out.npz
```

接下来将训练 GPT-2 模型。

步骤 9：训练 GPT-2 模型

我们将使用数据集训练 GPT-2 117M 模型。将编码后的数据集的名称发送给程序：

```
#@title Step 9:Training the Model
#Model saved after 1000 steps
import os # import after runtime is restarted
os.chdir("/content/gpt-2/src/")
!python train.py --dataset out.npz
```

运行该单元格时，它将一直训练直到你停止。这里只训练 1000 步。所以当训练超过 1000 步时，请停止它。模型将保存在 `/content/gpt-2/src/ checkpoint/run1`。可在笔记本的步骤 10A 检查这些文件的列表。

可通过双击单元格的 `run` 按钮来停止训练。然后将结束训练，并保存训练参数。

也可使用 `Ctrl + M` 在 1000 步之后停止训练模型。程序将停止训练并保存训练参数。将展示如图 D.1 所示的消息。

@title Step 9:Training the Model

Model saved after 1000 steps

图 D.1 自动保存训练好的 GPT-2 模型

程序使用 `/content/gpt-2/src/memory_saving_gradients.py` 和 `/content/gpt-2/src/accumulate.py` 来管理优化器和梯度。

`train.py` 包含一个完整的参数列表，可以调整以修改训练过程。我们一开始不要更改它们，先运行笔记本。然后，如果你愿意，可以尝试调整训练参数，看看是否可以获得更好的结果。

GPT-2 模型在训练过程中会生成一些样本，可以阅读它们。在我的 GPT-2 训练过程中，系统生成了一个很有启发性的样本：

```
The world is not a thing in itself, but is a representation of the world
in itself.
```

可以看到，AI 可以学习和理解人类的表达方式，并在某种程度上模仿和创造类似的表达。有趣！

接下来创建训练模型目录来继续我们的实验。

步骤 10：创建训练模型目录

本节将创建一个临时目录用于训练模型，存储我们需要的信息，并将其重命名以替换我们下载的 GPT-2 117M 模型的目录。

首先创建一个名为 `tgmodel` 的临时目录：

```
#@title Step 10: Creating a Training Model directory
#Creating a Training Model directory named 'tgmodel'
import os
run_dir = '/content/gpt-2/models/tgmodel'
if not os.path.exists(run_dir):
    os.makedirs(run_dir)
```

然后复制包含本附录步骤 9 中保存的训练参数的 `checkpoint` 文件：

```
#@title Step 10A: Copying training Files
!cp /content/gpt-2/src/checkpoint/run1/model-1000.data-00000-of-00001 /
content/gpt-2/models/tgmodel
!cp /content/gpt-2/src/checkpoint/run1/checkpoint /content/gpt-2/models/
tgmodel
!cp /content/gpt-2/src/checkpoint/run1/model-1000.index /content/gpt-2/
models/tgmodel
!cp /content/gpt-2/src/checkpoint/run1/model-1000.meta /content/gpt-2/
models/tgmodel
```

`tgmodel` 目录现在包含 GPT-2 模型的训练参数。

附录 C 的步骤 5 描述了这些文件的内容。

现在将从我们下载的 GPT-2 117M 模型中检索超参数和词表文件：

```
#@title Step 10B: Copying the OpenAI GPT-2 117M Model files
!cp /content/gpt-2/models/117M/encoder.json /content/gpt-2/models/tgmodel
```

```
!cp /content/gpt-2/models/117M/hparams.json /content/gpt-2/models/tgmodel
!cp /content/gpt-2/models/117M/vocab.bpe /content/gpt-2/models/tgmodel
```

tgmodel 目录现在包含了我们完整定制的 GPT-2 117M 模型。

最后一步是将我们下载的原始 GPT-2 模型重命名，并将模型命名为 117M:

```
#@title Step 10C: Renaming the model directories
import os
!mv /content/gpt-2/models/117M /content/gpt-2/models/117M_OpenAI
!mv /content/gpt-2/models/tgmodel /content/gpt-2/models/117M
```

至此我们已经训练完模型了，接下来让我们与模型互动吧！

步骤 11：生成无条件样本

现在将与使用了我们自定义数据集训练的 GPT-2 117M 模型进行交互。首先生成一个无条件样本(即不需要我们输入任何内容的样本)。然后，将输入一个上下文段落，以获得训练模型的条件文本补全响应。

首先运行一个无条件样本：

```
#@title Step 11: Generating Unconditional Samples
import os # import after runtime is restarted
os.chdir("/content/gpt-2/src")
!python generate_unconditional_samples.py --model_name '117M'
```

这里你不需要输入上下文句子，因为我们的模型可以是无条件样本生成器。

如果你要停止单元格，请双击单元格的 Run 按钮或按 Ctrl + M。

结果是随机的，但从语法角度看是有意义的。从语义角度看，结果并不那么有趣，因为我们没有提供上下文。但是，这个过程仍然是了不起的。它发明了一个帖子，写了一个标题并标上日期，发明了组织和地址，并生成一个主题，甚至想象了一个网页链接！

前几行令人难以置信：

```
Title: total_authority
Category:
Style: Printable
Quote:
Joined: July 17th, 2013
Posts: 0
Offtopic link: "Essential research, research that supports papers being
peer reviewed, research that backs up one's claims for design, research
that unjustifiably accommodates scientific uncertainties, and research
that persuades opens doors for science and participation in science",
href: https://groups.google.com/
search?q=Author%3APj&src=ieKZP4CSg4GVWDSJtwQczgTWQhAWBO7+tKwn0jzz7o6rP4l
Ey&s
sl=cTheory%20issue1&fastSource=posts&very=device
Offline
```

```
Joined: May 11th, 2014
Posts: 1729
Location: Montana AreaJoined: May 11th, 2014Posts: 1729Location: Montana
Posted: Fri Dec 26, 2017 9:18 pm Post subject: click
I. Synopsis of the established review group
The "A New Research Paradigm" and Preferred Alternative (BREPG) group lead
authors John Obi (Australian, USA and Chartered Institute of Tropical and
Climate Change Research), Marco Xiao (China and Department of Sociology/
Ajax, International Institute of Tropical and Climate Change Research,
Shanghai University) and Jackie Gu (US/Pacific University, Interselicitas
de NASA and Frozen Planet Research Research Center, Oak Ridge National
Laboratory). Dr. Obi states: "Our conclusions indicate that the existence
of the new peer reviewed asan-rubie study predisposes journal publishers
to read scientific publishers constantly to seek a consignment of, and
to be affiliated with, a large target certain of their persons. The
current practice of supplying books with peer review by commonly-overruled
specialists itself spreads from part to part, sufficient to spread the
impact of peer reviews by experts to all levels of the organization, so as
to increase the extent of exposure to the term of deviation that source
profiles or findings require".
```

无条件随机文本生成器的结果是有趣的，但难以令人信服。

步骤 12：交互式上下文和文本补全示例

现在将运行一个条件样本。我们输入的上下文将使模型按照我们的意愿进行思考，然后生成定制的段落来补全文本。

运行单元格并探索魔法：

```
#@title Step 12: Interactive Context and Completion Examples
import os # import after runtime is restarted
os.chdir("/content/gpt-2/src")
!python interactive_conditional_samples.py --temperature 0.8 --top_k 40
--model_name '117M'
```

如有必要，请花几分钟回顾一下附录 C 步骤 9 中的参数。现在程序提示我们输入上下文，如图 D.2。

prompt >>>



图 D.2 输入文本补全的上下文

输入与附录 C 步骤 9 同样的 Emmanuel Kant 的段落：

```
Human reason, in one sphere of its cognition, is called upon to
consider questions, which it cannot decline, as they are presented by
its own nature, but which it cannot answer, as they transcend every
faculty of the mind.
```

按 Enter 键生成文本，就像我们之前做的那样。同样，每次运行的输出都会不一样，尽管它是有结构和逻辑的，这使得 Transformer 很有吸引力。不过这次的结果与

附录 C 的结果相比，令人印象深刻。

我们看一下 GPT-2 模型生成的前几行：

```
"Now, what is the nature of the reason, except that it is of a kind and power of mind that is so called on to think, and to understand? The reason is the thing which apprehends, and seeks to understand, the objects of its minds, which apprehend and apprehend the things of itself. No one can speak of this reason without the understanding and the understanding of the human reason. But that knowledge and understanding which is the most natural of all the faculties, which, because it is understood by the human reason, is the most natural of all the faculties, is not without reason, but only without understanding. The human reason, who sees and knows the things which the natural sense, which, though in the natural sense, does not understand, is the source of every other knowledge which we understand. The reason is the thing which does not understand, but understands the things which we comprehend, and therefore, is not to be understood. I mean that the reason is the thing which does not understand, but knows only the things which we do understand..."
```

如果你想停止单元格，请双击单元格的 **run** 按钮或按 **Ctrl + M**。

哇！我怀疑任何人都看不出我们训练的 GPT-2 模型产生的文本补全与人类之间的区别。它也可能在每次运行时生成不同的输出。这可能是其中的一个输出，例如：

```
The first question, to which it is obliged to respond, is, how can it be true that a mind which is ignorant of the truth of any proposition can, or ought, to, accept any proposition which it has not given it? And this question has been so well answered that it is impossible for any man to believe in the existence of any mind which has never received any information.  
"It is not the mind of any man, as far as we can judge, which is the subject of any philosophical inquiry. It is the mind of the minds of men, in their opinion, which they consider the most to be their most important subject. And if they can see through this, they will see it, and they will understand it, and they will understand it."
```

我认为模型在哲学、推理和逻辑的抽象练习中可以超越人类！但是，它的局限性在于每次运行时生成文本会有所不同。因此，尽管看起来很出色，但它并不能满足我们在日常生活中的每一个需求。

我们可以从实验中得出一些结论：

- 经过良好训练的 **Transformer** 模型可以生成接近人类水平的文本补全。
- **GPT-2** 模型在复杂和抽象推理的文本生成上几乎可以达到人类水平。
- 文本上下文是一种有效的、通过展示预期结果对模型进行条件约束的方法。
- 文本补全是基于文本条件约束的文本生成(如果提供了上下文)。
- 尽管文本达到了人类水平，但并不意味着它能满足我们的每一个需求。它在局部很有趣，但整体上并不有效。

可以尝试输入一些条件文本上下文进行文本补全实验。你还可使用自己的数据来

训练模型。只需要将 `dset.txt` 文件的内容替换为自己的数据，然后看看会发生什么！

记住，我们训练的 GPT-2 模型会像人类一样做出反应。如果你输入的上下文很短、不完整、无趣或难懂，你将得到困惑或糟糕的结果。这是因为 GPT-2 期望我们能够给出最好的输入，就像在现实生活中一样！

参考资料

- OpenAI GPT-2 GitHub 代码存储库：<https://github.com/openai/gpt-2>
- N Shepperd GitHub 代码存储库：<https://github.com/nshepperd/gpt-2> 训练模型。
只需要将 `dset.txt` 文件的内容替换你自己的数据，然后看看会发生什么！

附录 E

练习题答案

第 1 章

1. 我们仍处于第三次工业革命中。(对|错)

错误。历史上的时代确实有重叠。然而，第三次工业革命主要是将世界数字化。第四次工业革命已经开始将万物互联：系统、机器、机器人、算法等等。

2. 第四次工业革命将实现万物互联。(对|错)

正确。这导致了越来越多以前需要人为干预的决策实现自动化。

3. 工业 4.0 程序员有时不需要进行 AI 开发。(对|错)

正确。在有些项目中，程序员可以直接使用在线 AI 服务。

4. 工业 4.0 程序员可能不得不从头开始实施 Transformer。(对|错)

正确。在一些项目中，不是所有的标准在线服务或 API 都能满足项目的需求。某些情况下，可能没有一个令人满意的解决方案。相反，程序员将不得不对模型进行大量定制并从头开始工作。

5. 没必要学习多个 Transformer 生态系统，只学习一个(例如 Hugging Face)就足够了。(对|错)

错误。有些公司因为各种原因决定了只能与 Google Cloud AI 或 Microsoft Azure AI 合作。只有部分公司使用 Hugging Face。

6. 即用型 Transformer API 可以满足所有需求。(对|错)

如果它有效，就是正确的。如果 Transformer 模型训练不好，就是错误的。

7. 公司会接受开发商最了解的 Transformer 生态系统。(对|错)

错误。公司可能会接受或不接受开发商的建议。因此，最好尽可能学习更多的基础知识。

8. 云 Transformer 已成为主流。(对|错)

正确。

9. Transformer 项目可以在笔记本电脑上运行。(对|错)

原型可以，但涉及数千个用户的项目则不行。

10. 对工业 4.0 AI 专家的灵活性要求会更高。(对|错)
正确。

第 2 章

1. NLP 转导可以对文本表示进行编码和解码。(对|错)
正确。NLP 是将文本转换为数值表示，对其进行处理，并将结果解码回文本的过程。
2. 自然语言理解(NLU)是自然语言处理(NLP)的一个子集。(对|错)
正确。
3. 语言建模算法根据输入序列生成可能的单词序列。(对|错)
正确。
4. Transformer 是带有 CNN 层的定制 LSTM。(对|错)
错误。Transformer 根本不包含 LSTM 或 CNN。
5. Transformer 不包含 LSTM 或 CNN 层。(对|错)
正确。
6. 注意力机制检查序列中的所有词元，而不仅仅是最后一个词元。(对|错)
正确。
7. Transformer 使用位置向量，而不是位置编码。(对|错)
错误。Transformer 使用位置编码。
8. Transformer 包含一个前馈神经网络。(对|错)
正确。
9. Transformer 解码器的掩码多头注意力组件能够防止算法解析当前序列的后续部分。
正确。
10. Transformer 可以比 LSTM 更好地分析长距离依赖性。(对|错)
正确。

第 3 章

1. BERT 是 Bidirectional Encoder Representations from Transformers 的缩写。(对|错)
正确。
2. BERT 框架由两步组成。第 1 步是预训练。第 2 步是微调。(对|错)
正确。
3. 微调 BERT 模型意味着从头开始训练参数。(对|错)
错误。BERT 微调使用预训练的参数进行初始化。
4. BERT 仅使用所有下游任务进行预训练。(对|错)
错误。

5. BERT 使用掩码语言建模(MLM)进行预训练。(对|错)

正确。

6. BERT 使用下一句预测 (NSP)进行预训练。(对|错)

正确。

7. BERT 使用数学函数进行预训练。(对|错)

错误。

8. 问答任务是一项下游任务。(对|错)

正确。

9. BERT 预训练模型不需要词元化。(对|错)

正确。

10. 微调 BERT 模型比预训练花费的时间更少。(对|错)

正确。

第 4 章

1. RoBERTa 使用字节级 BPE 编码词元分析器。(对|错)

正确。

2. 使用 Hugging Face 词元分析器将生成 merges.txt 和 vocab.json 文件。(对|错)

正确。

3. RoBERTa 不会使用词元类型的 ID。(对|错)

正确。

4. DistilBERT 有 6 层和 12 个头。(对|错)

正确。

5.具有 8000 万个参数的 Transformer 模型是一个很大的模型。(对|错)

错误。8000 万参数是一个小模型。

6. 我们无法训练词元分析器。(对|错)

错误。词元分析器是可以训练的。

7. 类 BERT 模型有 6 个解码器层。(对|错)

错误。BERT 包含 6 个编码器层，而不是解码器层。

8. 掩码语言建模(MLM)预测句子中掩码词元所包含的单词。(对|错)

正确。

9. 类 BERT 模型没有自注意力子层。(对|错)

错误。BERT 有自注意力层。

10. 数据整理器有助于反向传播。(对|错)

正确。

第 5 章

1. 机器智能使用与人类相同的数据进行预测。(对|错)

有正确也有错误。

正确。某些情况下，机器智能在处理大量数据以提取含义和执行各种任务方面超过人类，这对人类来说可能需要几个世纪的时间。

错误。对于 NLU，人类通过感官可获得更多信息。机器智能依赖于人类提供的所有类型的媒体。

2. 对于 NLP 模型，SuperGLUE 比 GLUE 更糟糕。(对|错)

正确。

3. BoolQ 期望一个二进制答案。(对|错)

正确。

4. WiC 意为上下文中的单词。(对|错)

正确。

5. 识别文本蕴涵 (RTE) 检测一个序列是否包含另一个序列。(对|错)

正确。

6. Winograd 模式预测动词拼写是否正确。(对|错)

错误。Winograd 模式主要适用于代词消歧。

7. Transformer 模型现在处于 GLUE 和 SuperGLUE 的顶部。(对|错)

正确。

8. 人类基准标准不会永远占据榜首。SuperGLUE 也只能提高 Transformer 模型超越人类基准标准的难度而已。(对|错)

正确。

9. Transformer 模型永远不会超过 SuperGLUE 人类基准标准。(对|错)

有正确也有错误。

错误。Transformer 模型已经超过了 GLUE 的人类基准，并且将来也会在 SuperGLUE 上做到同样的事情。

正确。随着我们在 NLU 领域的进展，我们将继续设定更高的基准标准。

10. Transformer 模型的变体优于 RNN 和 CNN 模型。(对|错)

正确。但在 AI 领域，你永远不知道未来会发生什么！也许会有超越 Transformer 的新模型出现。

第 6 章

1. 机器翻译现在已经超过了人类的基准。(对|错)

错误。机器翻译是最具挑战性的 NLP 机器学习任务之一。

2. 机器翻译需要大型数据集。(对|错)

正确。

3. 不需要使用相同的数据集来比较 Transformer 模型。(对|错)

错误。比较不同模型的唯一方法是使用相同的数据集。

4. BLEU 是法语中蓝色的意思，是一个 NLP 评估指标(对|错)

正确。BLEU 是双语评估替代分数的缩写，很容易记住。

5. 平滑技术增强了 BERT。(对|错)

正确。

6. 德语-英语跟英语-德语的机器翻译过程是一样的。(对|错)

错误。先表示成德语然后翻译成另一种语言的过程与先表示成英语然后翻译成另一种语言的过程是不同的，语言结构也不同。

7. 原始 Transformer 多头注意力层有两个头。(对|错)

错误。每个注意力子层有八个头。

8. 原始 Transformer 编码器有 6 层。(对|错)

正确。

9. 原始 Transformer 编码器有 6 层，但解码器只有 2 层。(对|错)

错误。有 6 个解码器层。

10. 可以不使用解码器来训练 Transformer。(对|错)

正确。BERT 的架构只包含编码器。

第 7 章

1. 零样本方法对参数进行一次训练。(对|错)

错误。一次训练都不需要。

2. 运行零样本时执行梯度更新。(对|错)

错误。

3. GPT 模型只有一个解码器堆叠。(对|错)

正确。

4. 在本地机器上训练 117M GPT 模型是不可能的。(对|错)

错误。我们在本章中训练了一个。

5. 不可能使用特定的数据集训练 GPT-2 模型。(对|错)

错误。我们在本章中训练了一个。

6. 无法对 GPT-2 模型进行条件处理以生成文本。(对|错)

错误。我们在本章中实现了这个功能。

7. GPT-2 模型可以分析输入的上下文并生成完整内容。(对|错)

正确。

8. 我们无法在少于 345 个 GPU 的机器上与 8M 参数的 GPT 模型进行交互。(对|错)

错误。我们在本章与这个大小的模型进行了交互。

9. 不存在具有 285 000 个 CPU 的超级计算机。(对|错)

错误。

10. 拥有数千个 GPU 的超级计算机是人工智能的游戏规则改变者。(对|错)

正确。得益于这一点，我们将能构建具有越来越多参数和连接的模型。

第 8 章

1. T5 模型像 BERT 模型那样只有编码器堆叠。(对|错)

错误。

2. T5 模型同时具有编码器和解码器堆叠。(对|错)

正确。

3. T5 模型使用相对位置编码，而不是绝对位置编码。(对|错)

正确。

4. 文本到文本模型仅适用于文本摘要。(对|错)

错误。

5. 文本到文本模型使用前缀来确定 NLP 任务的输入序列。(对|错)

正确。

6. T5 模型需要为每个 NLP 任务提供对应的超参数。(对|错)

错误。

7. 文本到文本模型的优点在于它们对所有 NLP 任务使用相同的超参数。(对|错)

正确。

8. T5 模型没有包含前馈神经网络。(对|错)

错误。

9. Hugging Face 是一个框架，能让人们更容易地使用 Transformer。(对|错)

正确。

10. OpenAI 的 Transformer 引擎是游戏规则的改变者。(对|错)

正确。OpenAI 已经推出了一系列可直接使用的引擎，如 Codex(将语言转换为代码)或 Davinci(通用引擎)。

第 9 章

1. 词元化词典包含一门语言中的所有单词。(对|错)

错误。

2. 预训练词元分析器可以对任何数据集进行编码。(对|错)

错误。

3. 最好在使用数据集之前对其进行检查。(对|错)

正确。

4. 从数据集中删除淫秽数据是一种很好的做法。(对|错)
正确。
5. 从数据集中删除歧视数据是一种很好的做法。(对|错)
正确。
6. 原始数据集有时可能会在噪声和有用内容之间建立关系。(对|错)
正确。
7. 标准的预训练词元分析器包含过去 700 年的英语词汇。(对|错)
错误。
8. 使用经过现代英语训练的词元分析器对数据进行编码时，若遇到古代英语可能会产生问题。(对|错)
正确。
9. 医学和其他类型的术语在使用现代英语训练的词元分析器编码数据时会产生问题。(对|错)
正确。
10. 控制由预训练词元分析器生成的编码数据的输出是一种很好的做法。(对|错)
正确。

第 10 章

1. 语义角色标注(SRL)是一项文本生成任务。(对|错)
错误
2. 谓语是名词。(对|错)
错误
3. 动词是谓语。(对|错)
正确
4. 论元可以描述谁在做某事。(对|错)
正确
5. 修饰词可以是副词。(对|错)
正确
6. 修饰词可以是位置。(对|错)
正确
7. 基于 BERT 的模型包含编码器和解码器堆叠。(对|错)
错误
8. 基于 BERT 的 SRL 模型具有标准输入格式。(对|错)
正确
9. Transformer 可解决任何 SRL 任务。(对|错)
错误

第 11 章

1. 经过训练的 Transformer 模型可以回答任何问题。(对|错)
错误
2. 问答不需要进一步研究。它是完美的。(对|错)
错误
3. 命名实体识别(NER)可以提供有用信息来生成有意义的问题。(对|错)
正确
4. 语义角色标注(SRL)无法提供有用信息来生成有意义的问题。(对|错)
错误
5. 问题生成器是生成问题的绝佳方式。(对|错)
正确
6. 实施问答系统需要精细的项目管理。(对|错)
正确
7. ELECTRA 模型具有与 GPT-2 相同的架构。(对|错)
错误
8. ELECTRA 模型具有与 BERT 相同的架构，但被训练为判别器。(对|错)
正确
9. NER 可以识别一个位置并将其标记为 I-LOC。(对|错)
正确
10. NER 可以识别一个人并将该人标记为 I-PER。(对|错)
正确

第 12 章

1. 不需要对 Transformer 进行情绪分析的预训练。(对|错)
错误
2. 一个句子总是肯定的或消极的，不可能是中立的。(对|错)
错误
3. 组合性原则意味着 Transformer 必须掌握句子的每个部分才能理解这个句子。
(对|错)
正确
4. RoBERTa-large 旨在改进 Transformer 模型的预训练过程。(对|错)
正确
5. Transformer 可以提供反馈，告知我们客户是否满意。(对|错)
正确
6. 如果产品或服务的情绪分析始终是负面的，这有助于我们做出适当的决策来改进服务。(对|错)

正确

7. 如果模型未能在任务上提供良好结果,则在更改模型前需要进行更多训练。(对|错)

正确

第 13 章

1. 被标注为假新闻的新闻总是假的。(对|错)

错误

2. 每个人都同意的新闻总是真的。(对|错)

错误

3. Transformer 可用于对推文运行情绪分析。(对|错)

正确

4. 可使用 DistilBERT 模型运行 NER 以从 Facebook 消息中提取出关键实体。(对|错)

正确

5. 可使用基于 BERT 的模型运行 SRL 来识别 YouTube 聊天中的关键动词。(对|错)

正确

6. 情绪反应是对假新闻自然的第一反应。(对|错)

正确

7 理性对待假新闻有助于澄清自己的立场。(对|错)

正确

8. 将 Transformer 与可靠的网站连接在一起可以帮助某人理解为什么一些新闻是假的。(对|错)

正确

9. Transformer 可以对可靠网站进行摘要,帮助我们理解一些被标注为假新闻的主题。(对|错)

正确

10. 如果你为了所有人的利益而使用 AI,可以改变世界。(对|错)

正确

第 14 章

1. BertViz 仅展示 BERT 模型最后一层的输出。(对|错)

错误。BERTViz 展示所有层的输出。

2. BertViz 展示 BERT 模型每一层的注意力头。(对|错)

正确。

3. BertViz 展示了词元之间的相互关系。(对|错)

正确。

4. LIT 像 BertViz 那样展示了注意力头的内部运作。(对|错)
错误。不过 LIT 进行非探测预测。
5. 探测是算法预测语言表示的一种方式。(对|错)
正确。
6. NER 是一项探测性任务。(对|错)
正确。
7. PCA 和 UMAP 是非探测性任务。(对|错)
正确。
8. LIME 是模型无关的。(对|错)
正确。
9. Transformer 层层深挖词元的关系。(对|错)
正确。
10. 可视化 Transformer 模型解释为可解释 AI 增加了一个新的维度。(对|错)
正确。

第 15 章

1. Reformer Transformer 模型不包含编码器。(对|错)
错误。Reformer Transformer 模型包含编码器。
2. Reformer Transformer 模型不包含解码器。(对|错)
错误。Reformer Transformer 模型包含编码器和解码器。
3. 输入逐层存储在 Reformer 模型中。(对|错)
错误。输入在每层都会重新计算，从而节省内存。
4. DeBERTa Transformer 模型解缠内容和位置。(对|错)
正确。
5. 在为项目选择最终 Transformer 模型之前，有必要测试数百个预训练 Transformer 模型。(对|错)
有正确有错误。可以尝试所有的模型，或者选择一个非常可靠的模型。
6. 最新的 Transformer 模型永远是最好的。(对|错)
有正确有错误。可以尝试所有的模型，或者选择一个非常可靠的模型。
7. 最好每个 NLP 任务有一个 Transformer 模型，而不是一个多任务 Transformer 模型对应多个 NLP 任务。(对|错)
有正确有错误。这是你需要做出的个人决定。风险评估是项目的关键方面。
8. Transformer 模型始终需要微调。(对|错)
错误。GPT-3 引擎是零样本模型，不需要微调。
9. OpenAI GPT-3 引擎可以执行广泛的 NLP 任务，而不需要微调。(对|错)
正确。

10. 在本地服务器上实现 AI 算法总是更好的。(对|错)
错误。这取决于你的项目。这是你需要进行的风险评估。

第 16 章

1. 不存在可以自动生成代码的 AI 助理。(对|错)
错误。例如，GitHub Copilot 现在已经投入使用。
2. AI 助理永远不会取代人类。(对|错)
有正确有错误。AI 将接管销售、支持、维护和其他领域的许多任务。然而，许多复杂的任务仍然需要人类干预。
3. GPT-3 引擎只能执行一项任务。(对|错)
错误。GPT-3 引擎可以完成各种各样的任务。
4. 可以使用 Transformer 训练推荐系统。(对|错)
正确。Transformer 已经从语言序列发展到许多领域的序列。
5. Transformer 只能处理语言。(对|错)
错误。Transformer 可以分析许多其他类型的序列。
6. Transformer 序列只能包含单词。(对|错)
错误。Transformer 序列可以包含其他许多类型。
7. 视觉信息提供者不等于 CNN。(对|错)
错误。Transformer 是深度神经网络，在计算机视觉领域可以与 CNN 相媲美。
8. 具有计算机视觉的 AI 机器人是不存在的。(对|错)
错误。例如，在军事应用中已经出现了具有计算机视觉的机器人。
9. 不可能自动生成 Python 源代码。(对|错)
错误。Microsoft 和 OpenAI 已经合作开发了一个可以与我们一起编写或为我们编写 Python 代码的 Copilot。
10. 我们也许有一天会成为机器人的副驾驶。(对|错)
这可能会变成真，也可能会变成假。这仍然是人类、机器人和机器人在不断增长的 AI 生态系统中面临的挑战。

第 17 章

1. GPT-4 有自主意识。(对|错)
错误。GPT-4 是一个数学算法。它不需要有感知能力来学习统计模式以完成各种任务。
2. ChatGPT 可以取代人类专家。(对|错)
错误。ChatGPT 可以根据其数据集生成结果。然而，它不能做出专业决策。
3. GPT-4 可为任何任务生成源代码。(对|错)
错误。GPT-4 可以为许多任务生成源代码。然而，对于复杂的问题，需要人类干预。

4. 高级提示工程是直观的。(对|错)

错误。高级提示工程已成为一种基于“对 Transformer 的深入了解”的技能。高级提示工程涉及构建知识库、使用多种类型的对象来完成 API，以及理解许多可用的模型。

5. 最先进的 Transformer 模型(也就是 GPT-4)是目前能够使用的最佳模型。(对|错)

错误。这就像选择一款汽车。最昂贵和最快的汽车不一定是你需要的。有些项目可能不需要最强大的 Transformer 模型，够用就好。

6. 使用 Transformer 开发应用程序不需要训练，因为直接使用像 GPT-4 这样的编程助手就可以完成工作了。(对|错)

错误。GPT-4 这样的编程助手只是助手。Transformer 模型有时可以编写完整的函数或程序。然而，对于更复杂的程序，还是需要人类干预。

7. GPT-4 将是最后一个 OpenAI Transformer 模型，因为它已达到人工智能的极限。(对|错)

错误。OpenAI 无疑会推出更好的模型，竞争对手也会如此。

参 考 资 料

第 1 章的参考资料

- Bommansani et al. 2021, On the Opportunities and Risks of Foundation Models, <https://arxiv.org/abs/2108.07258>
- Chen et al.,2021, Evaluating Large Language Models Trained on Code, <https://arxiv.org/abs/2107.03374>
- Microsoft AI: <https://innovation.microsoft.com/en-us/ai-at-scale>
- OpenAI: <https://openai.com/>
- Google AI: <https://ai.google/>
- Google Trax: <https://github.com/google/trax>
- AllenNLP: <https://allennlp.org/>
- Hugging Face: <https://huggingface.co/>

第 2 章的参考资料

- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Go-mez, Lukasz Kaiser, Illia Polosukhin, 2017, Attention Is All You Need, <https://arxiv.org/abs/1706.03762>
- Hugging Face Transformer 用法: <https://huggingface.co/transformers/usage.html>
- Tensor2Tensor(T2T)简介: https://colab.research.google.com/github/tensorflow/tensor2tensor/blob/master/tensor2tensor/notebooks/hello_t2t.ipynb?hl=en
- Manuel Romero notebook with link to explanations by Raimi Karim: <https://colab.research.google.com/drive/1rPk3ohrmVclqhH7uQ7qys4oznDdAhpzF>
- Google language research: <https://research.google/teams/language/>
- Hugging Face research: <https://huggingface.co/transformers/index.html>
- The Annotated Transformer: <http://nlp.seas.harvard.edu/2018/04/03/attention.html>
- Jay Alammr, The Illustrated Transformer: <http://jalammar.github.io/illustrated-transformer/>

第 3 章的参考资料

- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin, 2017, Attention Is All You Need: <https://arxiv.org/abs/1706.03762>
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova, 2018, BERT: Pretraining of Deep Bidirectional Transformers for Language Understanding: <https://arxiv.org/abs/1810.04805>
- Alex Warstadt, Amanpreet Singh, and Samuel R. Bowman, 2018, Neural Network Acceptability Judgments: <https://arxiv.org/abs/1805.12471>
- 语言可接受性语料库(CoLA): <https://nyu-mln.github.io/CoLA/>
- Hugging Face 模型相关文档:
 - https://huggingface.co/transformers/pretrained_models.html
 - https://huggingface.co/transformers/model_doc/bert.html
 - https://huggingface.co/transformers/model_doc/roberta.html
 - https://huggingface.co/transformers/model_doc/distilbert.html

第 4 章的参考资料

- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyano, 2019, RoBERTa: A Robustly Optimized BERT Pretraining Approach: <https://arxiv.org/abs/1907.11692>
- Hugging Face Tokenizer 文档: https://huggingface.co/transformers/main_classes/tokenizer.html?highlight=tokenizer
- The Hugging Face 参考笔记本: https://colab.research.google.com/github/huggingface/blog/blob/master/notebooks/01_how_to_train.ipynb
- Hugging Face 参考博客: https://colab.research.google.com/github/huggingface/blog/blob/master/notebooks/01_how_to_train.ipynb
- 关于 BERT 的更多信息: https://huggingface.co/transformers/model_doc/bert.html
- 关于 DistilBERT 的更多信息: <https://arxiv.org/pdf/1910.01108.pdf>
- 关于 RoBERTa 信息的另一个来源: https://huggingface.co/transformers/model_doc/roberta.html

- 关于 DistilBERT 的更多信息: https://huggingface.co/transformers/model_doc/distilbert.html

第 5 章的参考资料

- Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, Samuel R. Bowman, 2019, SuperGLUE: A Stickier Benchmark for General-Purpose Language Understanding Systems: <https://w4ngatang.github.io/static/papers/superglue.pdf>
- Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, Samuel R. Bowman, 2019, GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding
- Yu Sun, Shuohuan Wang, Yukun Li, Shikun Feng, Hao Tian, Hua Wu, Haifeng Wang, 2019, ERNIE 2.0: A Continual Pretraining Framework for Language Understanding: <https://arxiv.org/pdf/1907.12412.pdf>
- Melissa Roemmele, Cosmin Adrian Bejan, and Andrew S. Gordon, 2011, Choice of Plausible Alternatives: An Evaluation of Commonsense Causal Reasoning: <https://people.ict.usc.edu/~gordon/publications/AAAI-SPRING11A.PDF>
- Richard Socher, Alex Perelygin, Jean Y. Wu, Jason Chuang, Christopher D. Manning, Andrew Y. Ng, and Christopher Potts, 2013, Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank: https://nlp.stanford.edu/~socherr/EMNLP2013_RNTN.pdf
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Jamie Brew, 2019, HuggingFace's Transformers: State-of-the-art Natural Language Processing: <https://arxiv.org/abs/1910.0377>
- Hugging Face Transformer 使用手册: <https://huggingface.co/transformers/usage.html>

第 6 章的参考资料

- English-German BLEU scores with reference papers and code: <https://paperswithcode.com/sota/machine-translation-on-wmt2014-english-german>
- The 2014 Workshop on Machine Translation(WMT): <https://www.statmt.org/wmt14/translation-task.html>

- European Parliament Proceedings Parallel Corpus 1996-2011, parallel corpus French-English: <https://www.statmt.org/europarl/v7/fr-en.tgz>
- Jason Brownlee, Ph.D., How to Prepare a French-to-English Dataset for Machine Translation: <https://machinelearningmastery.com/prepare-french-english-dataset-machine-translation/>
- Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu, 2002, BLEU: a Method for Automatic Evaluation of Machine Translation: <https://aclanthology.org/P02-1040.pdf>
- Jason Brownlee, Ph.D., A Gentle Introduction to Calculating the BLEU Score for Text in Python: <https://machinelearningmastery.com/calculate-bleu-score-for-text-python/>
- Boxing Chen and Colin Cherry, 2014, A Systematic Comparison of Smoothing Techniques for Sentence-Level BLEU: <http://acl2014.org/acl2014/W14-33/pdf/W14-3346.pdf>
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin, 2017, Attention Is All You Need: <https://arxiv.org/abs/1706.03762>
- Trax 代码存储库: <https://github.com/google/trax>
- Trax 教程: <https://trax-ml.readthedocs.io/en/latest/>

第 7 章的参考资料

- OpenAI 和 GPT-3 引擎: <https://beta.openai.com/docs/engines/engines>
- BertViz GitHub Repository by Jesse Vig: <https://github.com/jessevig/bertviz>
- OpenAI 的超级计算机: <https://blogs.microsoft.com/ai/openai-azure-supercomputer/>
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin, 2017, Attention is All You Need: <https://arxiv.org/abs/1706.03762>
- Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, 2018, Improving Language Understanding by Generative Pre-Training: https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova, 2019, BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding: <https://arxiv.org/abs/1810.04805>
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya

- Sutskever, 2019, Language Models are Unsupervised Multitask Learners: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf
- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, Dario Amodei, 2020, Language Models are Few-Shot Learners: <https://arxiv.org/abs/2005.14165>
 - Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, Samuel R. Bowman, 2019, SuperGLUE: A Stickier Benchmark for General-Purpose Language Understanding Systems: <https://w4ngatang.github.io/static/papers/superglue.pdf>
 - Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, Samuel R. Bowman, 2019, GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding: <https://arxiv.org/pdf/1804.07461.pdf>
 - OpenAI GPT-2 GitHub 代码存储库: <https://github.com/openai/gpt-2>
 - N. Shepperd's GitHub 代码存储库: <https://github.com/nshepperd/gpt-2>
 - Common Crawl 数据集: <https://commoncrawl.org/big-picture/>

第 8 章的参考资料

- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, Peter J. Liu, 2019, Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer: <https://arxiv.org/pdf/1910.10683.pdf>
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin, 2017, Attention is All You Need: <https://arxiv.org/abs/1706.03762>
- Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani, 2018, Self-Attention with Relative Position Representations: <https://arxiv.org/abs/1803.02155>
- Hugging Face 框架和资源: <https://huggingface.co/>

- 美国蒙大拿州公司法: <https://corporations.uslegal.com/statecorporation-law/montana-corporation-law/#:~:text=Montana%20Corporation%20Law,carrying%20out%20its%20business%20activities>
- 托马斯·杰斐逊的美利坚合众国独立宣言: <https://www.gutenberg.org/ebooks/1>
- 美国权利法案: <https://www.gutenberg.org/ebooks/2>

第 9 章的参考资料

- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu, 2019, Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer: <https://arxiv.org/pdf/1910.10683.pdf>
- OpenAI GPT-2 GitHub 代码存储库: <https://github.com/openai/gpt-2>
- N. Shepperd GitHub 代码存储库: <https://github.com/nshepperd/gpt-2>
- Hugging Face 框架和资源: <https://huggingface.co/>
- U.S. Legal, Montana Corporate Laws: <https://corporations.uslegal.com/state-corporation-law/montana-corporation-law/#:~:text=Montana%20Corporation%20Law,carrying%20out%20its%20business%20activities>
- Martina Conte, Nadia Loy, 2020, Multi-cue kinetic model with non-local sensing for cell migration on a fibers network with chemotaxis: <https://arxiv.org/abs/2006.09707>
- 托马斯·杰斐逊《美国独立宣言》: <https://www.gutenberg.org/ebooks/1>
- 美国权利法案和相关文本: <https://www.gutenberg.org/ebooks/2>
- The Magna Carta: <https://www.gutenberg.org/ebooks/10000>
- The Critique of Pure Reason, The Critique of Practical Reason, and Fundamental Principles of the Metaphysic of Moral: <https://www.gutenberg.org>

第 10 章的参考资料

- Peng Shi and Jimmy Lin, 2019, Simple BERT Models for Relation Extraction and Semantic Role Labeling: <https://arxiv.org/abs/1904.05255>
- Allen AI 研究所: <https://allennlp.org/>
- Allen AI 研究所的语义角色标注资源: <https://demo.allennlp.org/semantic-role-labeling>

第 11 章的参考资料

- Allen AI 研究院: <https://allennlp.org/>
- Allen AI 研究院的 AI 阅读理解资源: <https://demo.allennlp.org/reading-comprehension>
- Kevin Clark, Minh-Thang Luong, Quoc V. Le, Christopher D. Manning, 2020, ELECTRA: Pretraining Text Encoders as Discriminators Rather Than Generators: <https://arxiv.org/abs/2003.10555>
- Hugging Face pipeline 介绍: https://huggingface.co/transformers/main_classes/pipelines.html
- GitHub Haystack 框架代码存储库: <https://github.com/deepset-ai/haystack/>

第 12 章的参考资料

- Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher Manning, Andrew Ng, and Christopher Potts, Recursive Deep Models for Semantic Compositionality over a Sentiment Treebank: https://nlp.stanford.edu/~socherr/EMNLP2013_RNTN.pdf
- Hugging Face 流水线、模型和文档:
 - https://huggingface.co/transformers/main_classes/pipelines.html
 - <https://huggingface.co/models>
 - <https://huggingface.co/transformers/>

第 13 章的参考资料

- Daniel Kahneman, 2013, Thinking, Fast and Slow
- Hugging Face Pipelines: https://huggingface.co/transformers/main_classes/pipelines.html
- Allen AI 研究所: <https://allennlp.org/>

第 14 章的参考资料

- BertViz: Jesse Vig, 2019, A Multiscale Visualization of Attention in the Transformer Model, 2019, <https://aclanthology.org/P19-3007.pdf>

- BertViz: <https://github.com/jessevig/BertViz>
- LIT, 情绪分析表示解释: <https://pair-code.github.io/lit/tutorials/sentiment/>
- LIT: <https://pair-code.github.io/lit/>
- Transformer visualization via dictionary learning: Zeyu Yun, Yubei Chen, Bruno A Olshausen, Yann LeCun, 2021, Transformer visualization via dictionary learning: contextualized embedding as a linear superposition of transformer factors, <https://arxiv.org/abs/2103.15949>
- 通过字典学习实现 Transformer 可视化: <https://transformervis.github.io/transformervis/>

第 15 章的参考资料

- Hugging Face Reformer: https://huggingface.co/transformers/model_doc/reformer.html?highlight=reformer
- Hugging Face DeBERTa: https://huggingface.co/transformers/model_doc/deberta.html
- Pengcheng He, Xiaodong Liu, Jianfeng Gao, Weizhu Chen, 2020, Decoding-enhanced BERT with Disentangled Attention: <https://arxiv.org/abs/2006.03654>
- Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby, 2020, An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale: <https://arxiv.org/abs/2010.11929>
- OpenAI: <https://openai.com/>
- William Fedus, Barret Zoph, Noam Shazeer, 2021, Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity: <https://arxiv.org/abs/2101.03961>
- Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, Ilya Sutskever, 2021, Learning Transferable Visual Models From Natural Language Supervision: <https://arxiv.org/abs/2103.00020>
- C7LIP: <https://github.com/openai/CLIP>
- Aditya Ramesh, Mikhail Pavlov, Gabriel Goh, Scott Gray, Chelsea Voss, Alec Radford, Mark Chen, Ilya Sutskever, 2021, Zero-Shot Text-to-Image Generation: <https://arxiv.org/abs/2102.12092>

- DALL-E: <https://openai.com/blog/dall-e/>

第 16 章的参考资料

- OpenAI GPT-3 平台: <https://openai.com>
- OpenAI 模型和引擎: <https://beta.openai.com/docs/engines>
- Vision Transformers: Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby, 2020, An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale: <https://arxiv.org/abs/2010.11929>
- 用于视觉 Transformer 的 JAX: <https://github.com/google/jax>
- GitHub Copilot: <https://copilot.github.com/>
- Facebook 元宇宙: <https://www.facebook.com/Meta/videos/577658430179350>
- Markov Decision Process (MDP), examples and graph: Denis Rothman, 2020, Artificial Intelligence by Example, 2nd Edition: https://www.amazon.com/Artificial-Intelligence-Example-advanced-learning/dp/1839211539/ref=sr_1_3?crid=238SF8FPU7BB0&keywords=denis+rothman&qid=1644008912&srefix=denis+rothman%2Caps%2C143&sr=8-3

第 17 章的参考资料

- OpenAI ChatGPT blog: <https://openai.com/blog/chatgpt>
- OpenAI blog introducing the ChatGPT and Whisper APIs: <https://openai.com/blog/introducing-chatgpt-and-whisper-apis>
- OpenAI blog introducing ChatGPT Plus: <https://openai.com/blog/chatgpt-plus>
- OpenAI 模型: <https://platform.openai.com/docs/models/>
- OpenAI GPT-4: <https://openai.com/product/gpt-4>
- OpenAI Codex deprecated: <https://openai.com/blog/openai-codex>
- Microsoft GitHub Copilot with Codex (March 2023): <https://github.com/features/copilot>
- Google Text-to-Speech: <https://pypi.org/project/gTTS/>